```
                    account=msg.account;
                    pin="";
                    interface_hardware.send(display_enter_pin());
                    state=&atm::getting_pin;
                }
                );
    }
    void getting_pin();
public:
    void run()          <-- 5
    {
        state=&atm::waiting_for_card;    <-- 6
        try
        {
            for(;;)
            {
                (this->*state)();    <-- 7
            }
        }
        catch(messaging::close_queue const&)
        {
        }
    }
};
```

As already mentioned, the implementation described here is grossly simplified from the real logic that would be required in an ATM, but it does give you a feel for the message-passing style of programming. There's no need to think about synchronization and concurrency issues, just which messages may be received at any given point and which messages to send. The state machine for this ATM logic runs on a single thread, with other parts of the system such as the interface to the bank and the terminal interface running on separate threads. This style of program design is called the *Actor model*—there are several discrete *actors* in the system (each running on a separate thread), which send messages to each other to perform the task at hand, and there's no shared state except that directly passed via messages.

Execution starts with the `run()` member function ❺, which sets the initial state to `waiting_for_card` ❻ and then repeatedly executes the member function representing the current state (whatever it is) ❼. The state functions are simple member functions of the `atm` class. The `waiting_for_card` state function ❶ is also simple: it sends a message to the interface to display a "waiting for card" message ❷ and then waits for a message to handle ❸. The only type of message that can be handled here is a `card_inserted` message, which you handle with a lambda function ❹. You could pass any function or function object to the `handle` function, but for a simple case like this, it's easiest to use a lambda. Note that the `handle()` function call is chained onto the `wait()` function; if a message is received that doesn't match the specified type, it's discarded, and the thread continues to wait until a matching message is received.

The lambda function itself just caches the account number from the card in a member variable, clears the current PIN, sends a message to the interface hardware to display something asking the user to enter their PIN, and changes to the "getting PIN" state. Once the message handler has completed, the state function returns, and the main loop then calls the new state function ❼.